

Week 3: Script Design and Pseudocode

For Machine Learning Automation

NMTAFE - Certificate IV Data Science and AI

From concept to implementation: Planning robust ML workflows

Learning Objectives

- Translate application requirements into clear pseudocode and logic flows
- Design multi-step ML automation workflows using structured thinking
- Connect pseudocode concepts to functioning Python and Bash scripts
- Apply industry-standard planning approaches to ML pipeline development

Building on Previous Weeks

Week 1: Python scripting fundamentals and ML workflow basics

Week 2: Bash automation and shell environment management

Week 3: Today - Designing and planning complete workflows

The Bridge: From knowing individual commands to architecting complete systems

"A well-designed script saves hours of debugging and maintenance"

Why Pseudocode in ML Engineering?

Industry Reality:

- **Complex ML pipelines** involve 10+ interconnected steps
- **Team collaboration** requires shared understanding of workflows
- **Debugging** is easier when logic is clearly documented
- **Maintenance** and updates rely on clear design documentation

Pseudocode Benefits:

- Language-agnostic planning
- Identifies potential issues before coding
- Facilitates team communication and code reviews

The ML Pipeline Design Process

1. Requirements Analysis

- What data do we need to process?
- What models need to be trained?
- Where should outputs be stored?
- What error conditions might occur?

2. Workflow Design

- Break complex tasks into manageable steps
- Identify dependencies between steps
- Plan for error handling and recovery

3. Implementation Planning

Pseudocode Fundamentals

Basic Structure Elements:

```
BEGIN program_name
  DECLARE variables

  WHILE condition DO
    action_steps
  END WHILE

  IF condition THEN
    action_steps
  ELSE
    alternative_steps
  END IF

  CALL function_name(parameters)

END program_name
```

Example: Data Processing Pipeline Pseudocode

```
BEGIN ml_data_pipeline
  DECLARE input_directory, output_directory, log_file

  // Initialize environment
  CREATE output_directory IF NOT EXISTS
  INITIALIZE logging TO log_file

  // Process each data file
  FOR each_file IN input_directory DO
    LOG "Processing " + file_name

    IF file_extension == ".csv" THEN
      data = LOAD_CSV(file_path)
      cleaned_data = CLEAN_DATA(data)

      IF data_is_valid(cleaned_data) THEN
        SAVE cleaned_data TO output_directory
        LOG "Success: " + file_name
      ELSE
        LOG "Error: Invalid data in " + file_name
      END IF
    END IF
  END FOR

  GENERATE_SUMMARY_REPORT()
END ml_data_pipeline
```

From Pseudocode to Python Implementation

Pseudocode:

```
FOR each_file IN input_directory DO
    IF file_extension == ".csv" THEN
        data = LOAD_CSV(file_path)
        cleaned_data = CLEAN_DATA(data)
    END IF
END FOR
```

Python Implementation:

```
import pandas as pd
import os

for filename in os.listdir(input_directory):
    if filename.endswith('.csv'):
        file_path = os.path.join(input_directory, filename)
        data = pd.read_csv(file_path)
        cleaned_data = clean_data(data)
```

From Pseudocode to Bash Implementation

Pseudocode:

```
FOR each_file IN directory D0
  IF file_size > threshold THEN
    COMPRESS file
    MOVE compressed_file TO archive
  END IF
END FOR
```

Bash Implementation:

```
#!/bin/bash
for file in /data/*.csv; do
  file_size=$(stat -f%z "$file" 2>/dev/null || stat -c%s "$file")
  if [ "$file_size" -gt 1000000 ]; then
    gzip "$file"
    mv "${file}.gz" /archive/
  fi
done
```

Designing ML Training Workflows

Complete Training Pipeline Pseudocode:

```
BEGIN pytorch_training_pipeline
  // Environment setup
  VERIFY gpu_availability
  SET environment_variables
  LOAD configuration_file

  // Data preparation
  IF raw_data_exists THEN
    data = PREPROCESS_DATA(raw_data_path)
    train_data, val_data = SPLIT_DATA(data)
  ELSE
    ERROR "Raw data not found"
    EXIT
  END IF

  // Model training
  model = INITIALIZE_MODEL(config)
  optimizer = SETUP_OPTIMIZER(model, config)

  FOR epoch = 1 TO max_epochs DO
    train_loss = TRAIN_EPOCH(model, train_data)
    val_loss = VALIDATE_EPOCH(model, val_data)

    LOG epoch, train_loss, val_loss

    IF val_loss < best_loss THEN
      SAVE_MODEL(model, "best_model.pt")
      best_loss = val_loss
    END IF
  END FOR
```

Error Handling and Recovery Design

Robust Pseudocode Structure:

```
BEGIN robust_ml_script
  TRY
    MAIN_WORKFLOW()
  CATCH FileNotFoundError
    LOG "Data file missing"
    SEND_ALERT_EMAIL()
    EXIT_WITH_CODE(1)
  CATCH OutOfMemoryError
    LOG "Insufficient memory"
    CLEANUP_TEMP_FILES()
    RESTART_WITH_SMALLER_BATCH()
  CATCH NetworkError
    LOG "Network connection failed"
    RETRY_WITH_BACKOFF(max_retries=3)
END TRY

  FINALLY
    CLEANUP_RESOURCES()
    SEND_COMPLETION_NOTIFICATION()
  END FINALLY
```

Cloud Deployment Workflow Design

Azure ML Deployment Pseudocode:

```
BEGIN azure_ml_deployment
  // Authentication and setup
  AUTHENTICATE_AZURE()
  SET resource_group = "ml-production"
  SET vm_size = "Standard_NC6"

  // Infrastructure provisioning
  IF vm_exists(vm_name) THEN
    START_VM(vm_name)
  ELSE
    CREATE_VM(resource_group, vm_name, vm_size)
    WAIT_FOR_VM_READY()
  END IF

  // Deployment
  COPY_CODE_TO_VM(local_path, remote_path)
  COPY_MODEL_TO_VM(model_path, remote_model_path)

  SSH_EXECUTE(vm_name, "pip install -r requirements.txt")
  SSH_EXECUTE(vm_name, "python deploy_model.py")

  // Health check
  IF service_health_check() == "OK" THEN
    LOG "Deployment successful"
    UPDATE_LOAD_BALANCER()
  ELSE
    LOG "Deployment failed"
    ROLLBACK_DEPLOYMENT()
```

Practical Exercise: Design Your ML Workflow

Scenario: Design a complete workflow for the following requirements:

1. **Input:** Daily CSV files with sensor data
2. **Processing:** Clean data, detect anomalies, train model
3. **Output:** Model predictions and performance reports
4. **Constraints:** Must handle missing files gracefully
5. **Monitoring:** Log all operations and send alerts on failures

Your Task: Write pseudocode for this complete workflow

Exercise Solution Framework

```
BEGIN sensor_anomaly_pipeline
  // TODO: Add initialization steps
  // TODO: Add data validation logic
  // TODO: Add model training workflow
  // TODO: Add prediction generation
  // TODO: Add error handling
  // TODO: Add monitoring and alerts
END sensor_anomaly_pipeline
```

Think Through:

- What happens if today's data file is missing?
- How do you handle corrupted data?
- When should the model be retrained?
- What metrics indicate success or failure?

Version Control Integration

Pseudocode for Git Workflow:

```
BEGIN ml_development_workflow
  // Daily development cycle
  GIT_PULL(main_branch)
  CREATE_FEATURE_BRANCH("experiment-" + date)

  // Development work
  MODIFY_CODE()
  RUN_TESTS()

  IF tests_pass THEN
    GIT_ADD_COMMIT("Add feature: " + description)
    GIT_PUSH(feature_branch)
    CREATE_PULL_REQUEST()
  ELSE
    LOG "Tests failed - fix before commit"
  END IF
END ml_development_workflow
```

Documentation Standards in Pseudocode

Well-Documented Pseudocode:

```
// Purpose: Process daily sales data for ML model training
// Author: Jordan Hill
// Date: 2024-07-15
// Dependencies: pandas, torch, azure-cli
// Input: /data/raw/sales_YYYYMMDD.csv
// Output: /models/sales_model_YYYYMMDD.pt

BEGIN daily_sales_model_training
    // Step 1: Data validation and loading
    VALIDATE_INPUT_FILE_EXISTS()
    data = LOAD_AND_VALIDATE_CSV()

    // Step 2: Feature engineering
    // Create time-based features (day_of_week, month, season)
    features = ENGINEER_FEATURES(data)

    // Step 3: Model training with hyperparameter optimization
    best_model = TRAIN_WITH_HYPEROPT(features)

    // Step 4: Model evaluation and deployment decision
    IF model_performance > threshold THEN
        DEPLOY_MODEL(best_model)
```

Common Design Patterns in ML Automation

1. ETL (Extract, Transform, Load) Pattern:

```
EXTRACT data FROM source  
TRANSFORM data USING business_rules  
LOAD data INTO destination
```

2. Pipeline Pattern:

```
data → preprocessing → feature_engineering → model_training → evaluation → deployment
```

3. Retry Pattern:

```
FOR attempt = 1 TO max_retries DO  
  TRY operation  
  IF success THEN BREAK  
  WAIT exponential_backoff(attempt)  
END FOR
```

Industry Case Study: Production ML Pipeline

Real-World Scenario: E-commerce company's recommendation system

Requirements:

- Process 1M+ user interactions daily
- Update models every 6 hours
- A/B test multiple model versions
- 99.9% uptime requirement

Design Approach:

1. Break into microservices
2. Design fault-tolerant data pipelines
3. Implement blue-green deployment

Microservice Architecture Pseudocode

```
BEGIN recommendation_system
  // Service 1: Data Ingestion
  PARALLEL
    STREAM user_interactions FROM kafka
    BATCH_PROCESS historical_data
  END PARALLEL

  // Service 2: Feature Engineering
  features = COMPUTE_USER_FEATURES(interactions)
  CACHE features IN redis

  // Service 3: Model Training
  IF training_schedule_due THEN
    model = TRAIN_MODEL(features)
    VALIDATE_MODEL(model, test_data)
    IF validation_score > threshold THEN
      DEPLOY_MODEL(model)
    END IF
  END IF

  // Service 4: Prediction API
  SERVE_PREDICTIONS via rest_api
END recommendation_system
```

Testing and Validation in Pseudocode

Test-Driven Development Approach:

```
BEGIN test_driven_ml_development
  // Unit tests
  FOR each_function DO
    CREATE_TEST_CASES(function)
    VERIFY_EXPECTED_OUTPUTS()
  END FOR

  // Integration tests
  TEST_END_TO_END_PIPELINE()
  VERIFY_DATA_QUALITY()
  VALIDATE_MODEL_OUTPUTS()

  // Performance tests
  MEASURE_EXECUTION_TIME()
  VERIFY_MEMORY_USAGE()
  TEST_CONCURRENT_LOADS()

  IF all_tests_pass THEN
    APPROVE_FOR_PRODUCTION()
  ELSE
    IDENTIFY_FAILURES()
    FIX_AND_RETEST()
```

Monitoring and Alerting Design

Comprehensive Monitoring Pseudocode:

```
BEGIN ml_monitoring_system
  // Performance monitoring
  TRACK model_accuracy OVER time
  MONITOR prediction_latency
  MEASURE throughput_rates

  // Infrastructure monitoring
  CHECK cpu_utilization
  MONITOR memory_usage
  VERIFY disk_space

  // Business metrics
  TRACK prediction_quality
  MEASURE user_engagement
  MONITOR revenue_impact

  // Alerting logic
  IF any_metric EXCEEDS threshold THEN
    SEND_ALERT(severity_level, metric_details)
    IF critical_failure THEN
      TRIGGER_AUTOMATED_ROLLBACK()
    END IF
  END IF
```

Transition to Implementation

Next Week Preview: We move from design to version control

- Git fundamentals for ML projects
- Collaborative development workflows
- Code review and documentation standards

Today's Skills Set You Up For:

- Clearer implementation planning
- Better team communication
- Reduced debugging time
- More maintainable code

Best Practices Summary

- ✓ **Start with pseudocode** before writing any actual code
- ✓ **Include error handling** in your initial design
- ✓ **Document assumptions** and dependencies clearly
- ✓ **Design for maintainability** and team collaboration
- ✓ **Plan monitoring** and alerting from the beginning

"Time spent in design saves exponential time in debugging"

Practical Assignment

Your Task: Create complete pseudocode for an ML automation workflow that:

1. **Monitors** a directory for new data files
2. **Validates** data quality automatically
3. **Trains** a model when sufficient new data exists
4. **Deploys** the model if it outperforms the current version
5. **Alerts** stakeholders of successes and failures

Deliverable: Well-documented pseudocode with error handling

Assessment Preparation

Portfolio Development:

- Today's pseudocode exercises form planning documentation
- Week 6 assessment requires evidence of design thinking
- Show progression from requirements → pseudocode → implementation
- Document decision-making process and alternatives considered

Industry Standards: Employers value developers who plan before coding

Key Takeaways

- ✓ **Pseudocode bridges** requirements and implementation
- ✓ **Planning prevents** common pitfalls and reduces debugging time
- ✓ **Documentation standards** are essential for team collaboration
- ✓ **Error handling design** is as important as the main workflow

"Great ML engineers design great systems before they build them"

Questions and Discussion

Reflection Questions:

- How might pseudocode help you communicate technical ideas to non-technical stakeholders?
- What aspects of ML workflows are most challenging to design?
- When would you choose a simple vs. complex error handling strategy?

Next Week: Git and version control for collaborative ML development

